T.C.
Sakarya University
Faculty of Computer and Information Science
Software Engineering
Operating Systems

Operating Systems Course Project

Students Information:

B221202050 Ahmet Hamza Tok
B211202056 Büşranur Alaftargil
B241210351 Damla Söylemez

May 2026

Contents

1. Introduction
2. Design approach
3. Synchronization design
4. Deadlock detection method
5. Experiments
6. Performance evaluation
7. Conclusion

Introduction

This project implements a multithreaded producer–consumer system using POSIX threads in C. The implementation demonstrates synchronization, bounded-buffer communication, deadlock detection, logging, and runtime performance analysis.

Unlike the classical producer–consumer problem that uses a single producer and consumer, this system supports multiple producers, multiple consumers, named buffers, and pipeline workers that consume from one buffer while producing into another. This structure allows the simulation of task queues and staged processing pipelines.

Synchronization is implemented with mutexes and condition variables. In addition, the system includes a monitoring thread for deadlock detection using a wait-for graph approach. Runtime statistics such as throughput, blocking time, waiting time, deadlock frequency, and CPU utilization are also recorded.

The project satisfies the assignment requirements including concurrent execution, bounded shared buffers, synchronization support, deadlock monitoring, configuration-based topology, logging, and performance experiments.

Design Approach

The project implements a modular and scalable multi-threaded architecture designed to simulate complex data flow scenarios. The design is centered around three fundamental pillars:

- Dynamic Resource Mapping: Instead of hard-coded structures, the system uses a Configuration Parser to build a network of named buffers and threads at runtime. This allows for the flexible definition of buffer capacities and thread roles (Producer, Consumer, or Pipeline Worker) via external config files.
- Thread Context Abstraction: Every thread is decoupled from the main logic using a Thread Context structure. This structure encapsulates all operational data, such as timing intervals and buffer references, allowing the same generic code to handle diverse tasks depending on its assigned context.
- Decentralized Synchronization: To maximize concurrency, each named buffer operates as an independent synchronization unit. By utilizing localized POSIX Mutexes and Condition Variables (not_full, not_empty), the design ensures that threads only block when their specific resource is unavailable, preventing global system bottlenecks.
- Active Graph-Based Monitoring: The system incorporates a dedicated Deadlock Monitor that maintains a real-time state of the simulation. By tracking "held" and "requested" resources for every thread, it constructs a Wait-for Graph to proactively detect circular dependencies.
- Graceful Termination: Reliability is ensured through a controlled shutdown protocol. Upon reaching the runtime limit, a global broadcast signal forces all sleeping threads to wake up and exit cleanly, ensuring all allocated resources are released.

Synchronization design

The implementation is divided into separate modules responsible for configuration parsing, buffer management, producer and consumer logic, deadlock detection, metrics, and logging.

The runtime state is represented through a shared simulation_t structure containing: shared buffers runtime metrics synchronization resources

Report logger instance monitoring information Each thread receives a thread_context_t structure storing timing configuration, identity, and related input/output buffers. This makes the system configurable and avoids hardcoded dependencies.

Named Buffers and Pipelines

The system supports multiple buffers such as A and B . Producers insert items into buffers while consumers remove items. Some consumers also forward processed items into another buffer, creating pipeline stages.

Example: P1>A A>C1>B B>C3>A A>C9

This configuration creates a cyclic processing pipeline where some consumers transform data between buffers. 2.2 Shutdown Strategy The simulation runs for a fixed duration. After the configured runtime expires, the main thread sets a stop flag and broadcasts condition variables so blocked threads can wake up and terminate cleanly.

Synchronization Design

Synchronization is implemented using POSIX mutexes and condition variables. Each buffer maintains its own mutex together with not_full and condition variables.

Producer threads:

- wait for the production interval
- lock the buffer
- wait if the buffer is full
- insert an item
- signal consumers
- unlock the buffer

Consumer threads perform the reverse operation by waiting on empty buffers, consuming items, and signaling producers afterward.

Protected critical data includes queue indices, counters, and stored items. Mutex protection prevents race conditions such as simultaneous modification of queue state.

The system also records synchronization events including:

- lock acquisition and release
- waiting operations
- signaling operations
- blocking events

This logging support improves debugging and visibility of concurrent execution.

Deadlock Detection Method

In this project, deadlock handling is implemented with a dedicated monitor thread that periodically inspects the states of all producer and consumer threads. The monitor checks whether a thread is actively running or waiting for a resource. To avoid treating very short blocking events as deadlocks, the implementation uses an internal timeout threshold before a waiting thread is included in deadlock analysis.

Deadlock simulation is controlled by the "deadlock_start_delay_ms" configuration value. Before this delay expires, producers and consumers run normally. After the delay, they start acquiring two auxiliary locks in opposite order. Producers request "ResourceA" first and then "ResourceB", while consumers request "ResourceB" first and then "ResourceA". Under contention, this opposite ordering can satisfy circular wait and hold-and-wait conditions, creating a realistic deadlock scenario. The parameter "aux_lock_hold_ms" keeps the first lock for a short period before requesting the second one, increasing the probability of overlap between competing threads.

For detection, the monitor constructs a wait-for graph (WFG) from the collected waiting information. In this graph, nodes represent threads and directed edges represent dependencies, meaning that one thread is waiting for a resource held by another. After building the graph, a DFS-based cycle detection step is applied. If a cycle is found, the system reports a deadlock event in the logs and increments deadlock-related counters in the metrics module.

The configuration value "stop_on_deadlock" determines whether the simulation should stop immediately after a deadlock is detected. If it is enabled, the system requests all worker threads to stop and then prints the final metrics. If it is disabled, the simulation continues until the configured runtime ends.

Experiments

Several experiments were performed to evaluate synchronization behavior and performance

Metric	Low Load	High Load	Circular Dedendency	Bottleneck	Deadlock (delay=10000)	Deadlock (delay=3000)
Runtime	**60.5**	**60.16**	**60.24**	**60.25**	**60.27**	**4.08**
Produced	**7619**	**33539**	**2710**	**3792**	**560**	**167**
Consumed	**7618**	**33529**	**2710**	**3792**	**550**	**157**
Throughput	**126.86**	**557.33**	**44.98**	**62.94**	**9.12**	**38.48**
Avarage Waiting Time	**0.09**	**3.24**	**205.50**	**142.89**	**578.85**	**172.28**
Producer Blocking Time	**0**	**1178**	**0**	**0**	179653	**11628**
Consumer Blocking Time	**143**	**648**	**558947**	**543256**	**132345**	**14214**
Cpu Utilization	**0.34**	**1.47**	**0.15**	**0.21**	**0.04**	**0.16**
Deadlock Detections	0	0	0	0	2	2
Deadlock Frequency	0	0	0	0	0.0332	0.4902

Performance Evaluation

The implementation measures throughput, waiting times, blocking durations, deadlock frequency, and CPU utilization. Throughput is calculated as:
throughput = total_consumed / runtime_sec

High-load scenarios produced the highest throughput, while bottleneck and circular dependency experiments reduced efficiency. Deadlock frequency is computed as:

deadlock_frequency = deadlock_count / runtime_sec

Results showed that workload structure strongly affects contention, blocking time, and throughput behavior

Conclusion

This project successfully implements a configurable multithreaded producer consumer system with synchronization control, bounded buffers, deadlock monitoring, logging, and runtime performance analysis.

The implementation demonstrates important operating systems concepts including:
concurrent thread coordination
mutex-based synchronization
bounded-buffer communication
pipeline processing
deadlock detection
workload-sensitive performance behavior.

Experimental results confirmed that synchronization structure and workload distribution directly influence throughput, waiting time, and contention levels.
Overall, the project provides a practical demonstration of concurrency and synchronization mechanisms commonly studied in operating systems courses.